第16小时

层次式架构设计理论与实践

16.0　章节考点分析

第 16 小时主要学习层次式架构设计理论与实践。根据考试大纲，本小时知识点会涉及单选题型（约占 2～5 分）和案例题（25 分），本小时内容偏重于方法的掌握和应用，根据以往全国计算机技术与软件专业技术资格（水平）考试的出题规律，概念知识的考查内容多数来源于实际应用，还需要灵活运用相关知识点。

本小时知识架构如图 16.1 所示。

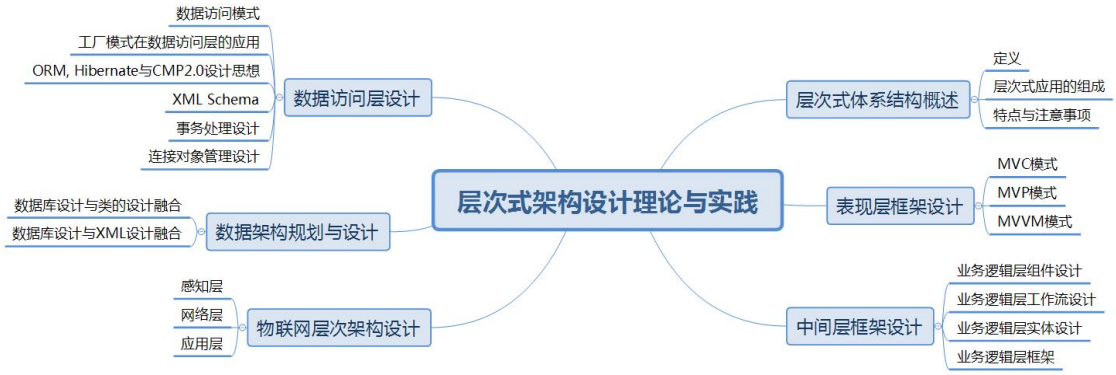

图 16.1　本小时知识架构图

【导读小贴士】

层次式架构是软件体系结构设计中最为常用的一种架构形式，它为软件系统提供了一种在结构、行为和属性方面的高级抽象，其核心思想是将系统组成为一种层次结构，每一层为上层服务，

并作为下层客户。本章重点介绍了层次式架构中的表现层、中间层、访问层和数据层的体系结构设计技术，给出了层次式架构的案例分析。

16.1 层次式体系结构概述

【基础知识点】

1. 定义

软件体系结构为软件系统提供了结构、行为和属性的高级抽象，由构成系统的元素描述这些元素的相互作用、指导元素集成的模式以及这些模式的约束组成。层次式体系结构设计是一种常见的架构设计方法，它将系统组成为一个层次结构，每一层为上层服务，并作为下层客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见。层次式体系结构的每一层最多只影响两层，同时只要给相邻层提供相同的接口，也允许每层用不同的方法实现，这种方式也为软件重用提供了强大的支持。

2. 层次式应用的组成

大部分的应用会分成表现层（或称为展示层）、中间层（或称为业务层）、访问层（或称为持久层）和数据层，如图 16.2 所示。

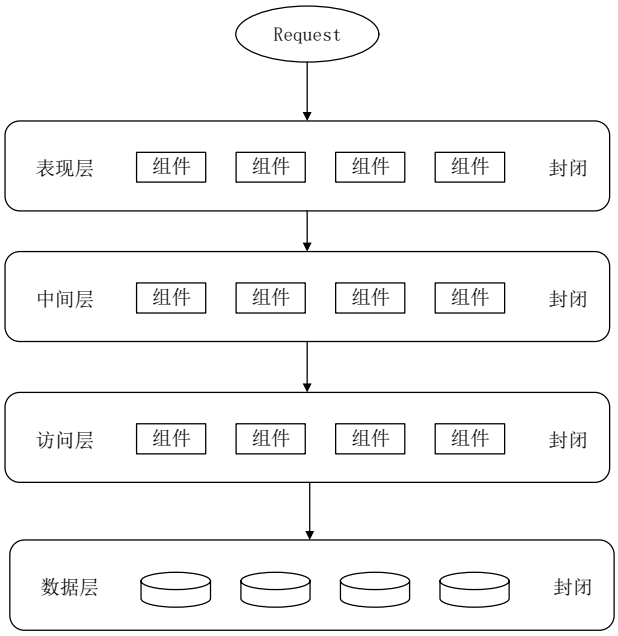

图 16.2 常用的层次式架构

3. 特点与注意事项

采用分层架构设计的一个特点就是关注点分离。每层中的组件只负责本层的逻辑，组件的划分

也很容易明确组件的角色和职责，比较容易开发、测试、管理和维护。层次式体系结构是一个可靠的通用的架构，但是设计时要注意以下两点：

1）容易成为污水池反模式（Architecture Sinkhole Anti-patter）：请求流简单地穿过几个层，每层里面基本没有做任何业务逻辑，或者做了很少的业务逻辑。比如一些 Java EE 例子，业务逻辑层只是简单地调用了持久层的接口，本身没有什么业务逻辑。

2）分层架构可能会让应用变得庞大。

16.2 表现层框架设计

【基础知识点】

1. MVC（Model-View-Controller）模式

MVC 是一种软件设计模式。MVC 把一个应用的输入、处理、输出流程按照视图、控制、模型的方式进行分离，形成了控制器、模型、视图 3 个核心模块。其中：

（1）控制器（Controller）：接受用户的输入，并调用模型和视图去完成用户的需求。

（2）模型（Model）：应用程序的主体部分，表示业务数据和业务逻辑。

（3）视图（View）：用户看到并与之交流的界面。

三者协作关系如图 16.3 所示。

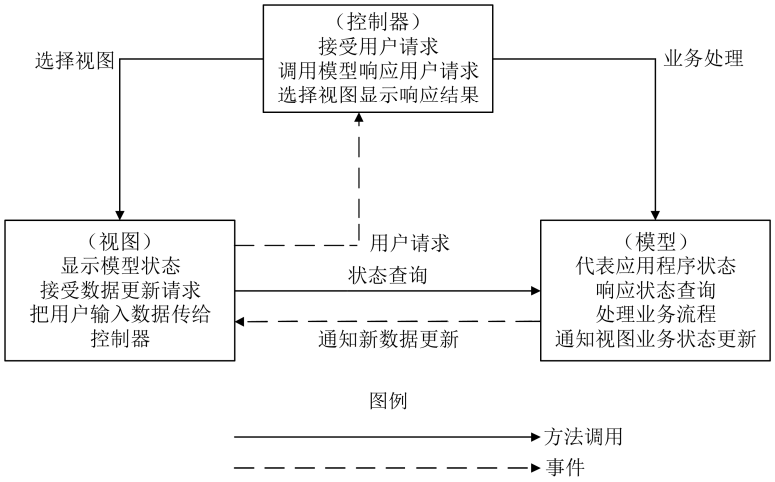

图 16.3 MVC 三者协作关系图

使用 MVC 模式来设计表现层，可以有以下的优点：

（1）允许多种用户界面的扩展。

（2）易于维护。

（3）易于构建功能强大的用户界面。

（4）增加应用的可拓展性、强壮性、灵活性。

2. MVP（Model-View-Presenter）模式

在 MVP 模式中 Model 提供数据，View 负责显示，Controller/Presenter 负责逻辑的处理。MVP 不仅仅避免了 View 和 Model 之间的耦合，还进一步降低了 Presenter 对 View 的依赖。MVP 设计模式如图 16.4 所示。

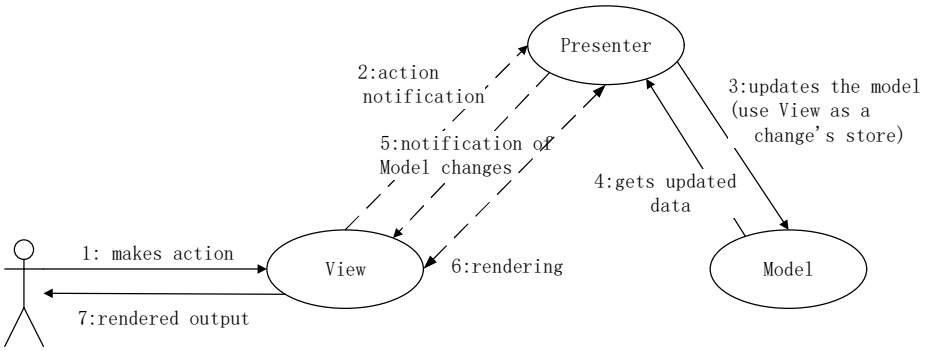

图 16.4　MVP 设计模式

使用 MVP 模式来设计表现层，可以有以下的优点：

（1）模型与视图完全分离，可以修改视图而不影响模型。

（2）所有的交互都发生在一个地方——Presenter 内部，因此可以更高效地使用模型。

（3）可以将一个 Presenter 用于多个视图，而不需要改变 Presenter 的逻辑。因为视图的变化总是比模型的变化频繁。

（4）如果把逻辑放在 Presenter 中，就可以脱离用户接口来测试这些逻辑（单元测试）。

3. MVVM（Model-View-View Model）模式

MVVM 和 MVC、MVP 类似，主要目的都是为了实现视图和模型的分离。不同的是 MVVM 中，View 与 Model 的交互通过 ViewModel 来实现，也就是 View 和 Model 不能直接通信，两者的通信只能通过 ViewModel 来实现。ViewModel 是 MVVM 的核心，通过 DataBinding 实现 View 与 Model 之间的双向绑定，其内容包括数据状态处理、数据绑定及数据转换。MVVM 流程设计模式如图 16.5 所示。

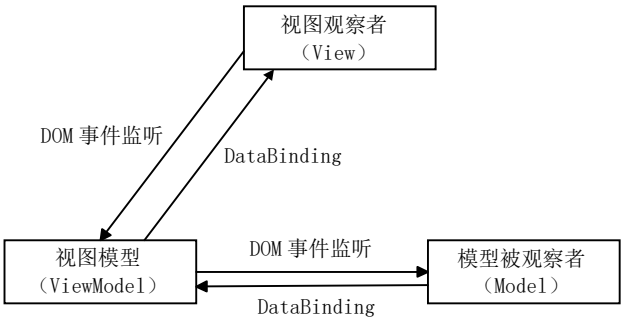

图 16.5　MVVM 流程设计模式

16.3 中间层框架设计

【基础知识点】

1. 业务逻辑层组件设计

业务逻辑层组件分为接口和实现类两个部分。接口用于定义业务逻辑组件，定义业务逻辑组件必须实现的方法是整个系统运行的核心。通常按模块来设计业务逻辑组件，每个模块设计一个业务逻辑组件，并且每个业务逻辑组件以多个数据访问对象（Data Access Object，DAO）组件作为基础，从而实现对外提供系统的业务逻辑服务。

2. 业务逻辑层工作流设计

工作流管理联盟（Workflow Management Coalition，WFMC）将工作流定义为：业务流程的全部或部分自动化，在此过程中，文档、信息或任务按照一定的过程规则流转，实现组织成员间的协调工作以达到业务的整体目标。工作流参考模型如图 16.6 所示。

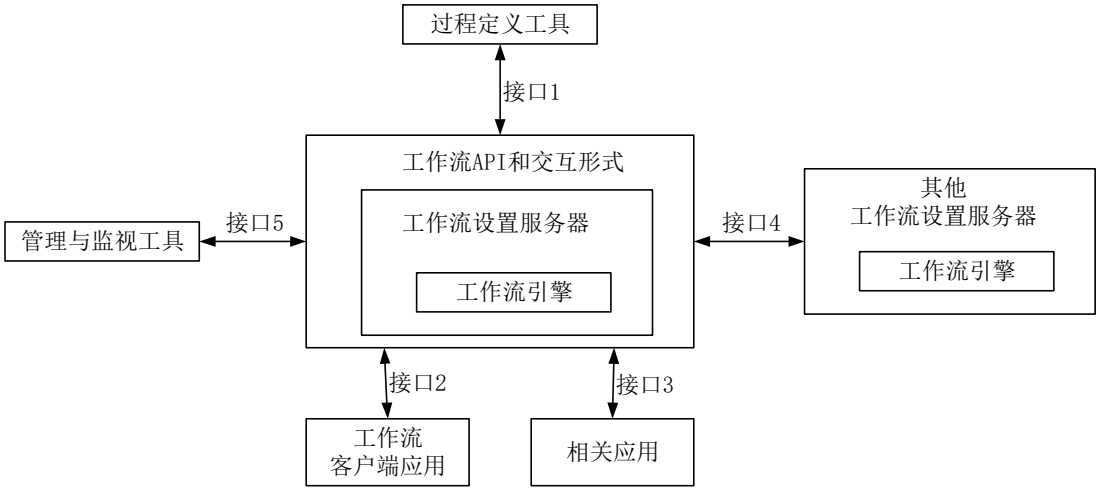

图 16.6　工作流参考模型

3. 业务逻辑层实体设计

逻辑层实体提供对业务数据及相关功能（在某些设计中）的状态编程访问。业务逻辑层实体可以使用具有复杂架构的数据来构建，这种数据通常来自数据库中的多个相关表。业务逻辑层实体数据可以作为业务过程的部分 I/O 参数传递。业务逻辑层实体是可序列化的，以保持它们的当前状态。

4. 业务逻辑层框架

业务逻辑框架位于系统架构的中间层，是实现系统功能的核心组件。采用容器的形式，便于系统功能的开发、代码重用和管理。在业务容器中，业务逻辑是按照 Domain Model-Service-Control 思想来实现的。其中：

1）Domain Model 是仅仅包含业务相关的属性的领域层业务对象。

2）Service 是业务过程实现的组成部分，是应用程序的不同功能单元，通过在这些服务之间定义良好的接口和契约联系起来。

3）Control 服务控制器，是服务之间的纽带，不同服务之间的切换就是通过它来实现的。

16.4 数据访问层设计

【基础知识点】

1. 数据访问模式

数据访问模式有 5 种，分别是：在线访问、Data Access Object、Data Transfer Object、离线数据模式、对象/关系映射（Object/Relation Mapping）。

（1）在线访问：最常用的方式。访问占用一个数据库连接，读取数据，每个数据库操作都会通过这个连接不断地与后台的数据源进行交互。

（2）Data Access Object：DAO 是标准 J2EE 设计模式，这种方式将底层数据访问操作与高层业务逻辑分离开。一个典型的 DAO 实现通常会有一个 DAO 工厂类、一个 DAO 接口、一个实现了 DAO 接口的具体类、数据传输对象。

（3）Data Transfer Object：DTO 属于 EJB 设计模式之一。DTO 是一组对象或容器，需要跨越不同的进程或是网络的边界来传输数据。

（4）离线数据模式：离线数据模式是以数据为中心，数据从数据源获取之后，将按照某种预定义的结构存放在系统中，成为应用的中心。这种方式对数据的各种操作独立于各种与后台数据源之间的连接或是事务。

（5）对象/关系映射：这种方式利用工具或平台能够帮助将应用程序中的数据转换成关系型数据库中的记录；或是将关系数据库中的记录转换成应用程序中代码便于操作的对象。

2. 工厂模式在数据访问层的应用

工厂模式定义一个用于创建对象的接口，让子类决定实例化哪一个类。工厂方法使一个类的实例化延迟到其子类。这里可能会处理对多种数据库的操作，因此，需要首先定义一个操纵数据库的接口，然后根据数据库的不同，由类工厂决定实例化哪个类。

3. ORM，Hibernate 与 CMP2.0 设计思想

ORM（Object-Relation Mapping）在关系型数据库和对象之间作一个映射，这样，在具体操纵数据库时，就不需要再去和复杂的 SQL 语句打交道，只要像平时操作对象一样操作即可。Hibernate 是一个功能强大，可以有效地进行数据库数据到业务对象的 O/R 映射方案。Hibernate 推动了基于普通 Java 对象模型，用于映射底层数据结构的持久对象的开发。

4. XML Schema

XML Schema 用来描述 XML 文档合法结构、内容和限制，提供丰富的数据类型。

5. 事务处理设计

事务必须服从 ISO/IEC 所制定的 ACID 原则。ACID 是原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持久性（Durability）的缩写。事务的原子性表示事务执行过程中的任何失败都将导致事务所做的任何修改失效。一致性表示当事务执行失败时，所有被该事务影响的数据都应该恢复到事务执行前的状态。隔离性表示在事务执行过程中对数据的修改，在事务提交之前对其他事务不可见。持久性表示已提交的数据在事务执行失败时，数据的状态都应该正确。

6. 连接对象管理设计

建立一个数据库连接池，提供一套高效的连接分配、使用策略，保证了数据库连接的有效复用。

16.5 数据架构规划与设计

【基础知识点】

（1）数据库设计与类的设计融合。对类和类之间关系的正确识别是数据模型的关键所在。好模型的目标是将工程项目整个生存期内的花费减至最小，同时也会考虑到随时间的推移系统将可能发生的变化，因而设计时也要考虑能适应这些变化。

（2）数据库设计与 XML 设计融合。XML 文档的存储方式有两种：基于文件的存储方式和数据库存储方式。

16.6 物联网层次架构设计

【基础知识点】

（1）感知层：用于识别物体、采集信息。感知层包括二维码标签和识读器、RFID 标签和读写器、摄像头、GPS、传感器、M2M 终端、传感器网关等，主要功能是识别对象、采集信息，与人体结构中皮肤和五官的作用类似。

（2）网络层：用于传递信息和处理信息。网络层包括通信网与互联网的融合网络、网络管理中心、信息中心和智能处理中心等。网络层将感知层获取的信息进行传递和处理，类似于人体结构中的神经中枢和大脑。

（3）应用层：实现广泛智能化。应用层是物联网与行业专业技术的深度融合，结合行业需求实现行业智能化，这类似于人们的社会分工。

16.7 练习题

1．软件体系结构为软件系统提供了（　　）的高级抽象，由构成系统的元素描述这些元素的相互作用、指导元素集成的模式以及这些模式的约束组成。

A．继承、多态、实现　　B．关联、扩展、泛化

C．结构、行为、属性　　D．构件定义、访问方式、组织部署

解析：软件体系结构为软件系统提供了结构、行为和属性的高级抽象，由构成系统的元素描述这些元素的相互作用、指导元素集成的模式以及这些模式的约束组成。

答案：C

2．MVC 模式是一种目前广泛流行的软件设计模式。近年来，随着 Java EE 的成熟，MVC 成为了 Java EE 平台上推荐的一种设计模式。MVC 强制性地把一个应用的（　　）流程进行分离，形成了控制器、模型、视图三个核心模块。

A．启动、运行、结束　　B．输入、处理、输出

C．前端/客户端、服务端、数据库　　D．接受请求、处理请求、返回请求

解析：MVC 模式是一种目前广泛流行的软件设计模式。近年来，随着 Java EE 的成熟，MVC 成为了 Java EE 平台上推荐的一种设计模式。MVC 强制性地把一个应用的输入、处理、输出流程进行分离，形成了控制器、模型、视图三个核心模块。

答案：B

3．工作流管理联盟（Workflow Management Coalition）将工作流定义为：业务流程的全部或部分自动化，在此过程中，文档、信息或任务按照一定的过程规则流转，实现组织成员间的协调工作以达到业务的整体目标。工作流参考模型包括的组件是（　　）。

A．过程定义工具、工作流引擎、工作流客户端应用、相关应用、管理与监视工具

B．工作流定义工具、工作流引擎、工作流客户端应用、相关应用、管理与监视工具

C．工作流定义工具、工作流引擎、工作流客户端应用、工作流 API、管理与监视工具

D．过程定义工具、工作流引擎、工作流客户端应用、工作流 API、管理与监视工具

解析：工作流参考模型包括的组件是过程定义工具、工作流引擎、工作流客户端应用、相关应用、管理与监视工具。

答案：A

4．事务必须服从 ISO/IEC 所制定的 ACID 原则。关于 ACID，以下说法错误的是（　　）。

A．事务的原子性表示事务执行过程中的任何失败都将导致事务所做的任何修改失效

B．一致性表示当事务执行失败时，所有被该事务影响的数据都应该恢复到事务执行前的状态

C．隔离性表示在事务执行过程中对数据的修改，在事务提交之后对其他事务不可见

D．持久性表示已提交的数据在事务执行失败时，数据的状态都应该正确

解析：隔离性表示在事务执行过程中对数据的修改，在事务提交之“前”对其他事务不可见。

答案：C

5．物联网的感知层用于识别物体、采集信息。下列（　　）不属于感知层设备。

A．摄像头　　B．GPS　　C．扫描仪　　D．指纹

解析：感知层主要功能是识别对象、采集信息，与人体结构中皮肤和五官的作用类似。但指纹是人的特征属性，不是感知层设备。

答案：D